

TECHNICAL REPORT

An open-source cross-check against Spike while the ImperasDV flow is relaunched

openhwgroup/cv32e40p

Document	RVX-CV32E40P-TRL5-05
Record date	2026-10-03
Prepared by	Rivoryxa Technologies
Classification	Public technical note. Not a verification sign-off.
Website	www.rivoryxatechnologies.com

1 Answer

While the ImperasDV-based suite is being relaunched, we run a cross-check that uses only open-source tools:

- Verilator for the RTL;
- Spike as the reference model;
- the riscv-dv Python generator for random programs.

It compares every retired instruction of the design, read from its RVFI trace, with Spike. This includes runs with random interrupts, debug requests and bus stalls.

On the TRL5 candidate configurations (I0, F0-F2, Z0-Z2) at RTL 6033d2b, it finds one real design divergence: #1060. Every other difference has an explained cause, listed below.

This is a stopgap that runs today, not a replacement for ImperasDV. It has known blind spots (see Limits). The most important is that the debug-entry cause is taken from the design, not checked independently.

2 The question

At the CVE4 meeting, the relaunch of the original verification suite (Dolphin's, built on Imperas and Synopsys tools) was reported as blocked:

- the testbench-to-ImperasDV connection has plugin problems;
- the current Imperas model must be checked against the one frozen two years ago.

Our question was: what can be checked today, with no licence, while that is resolved, and how much of it can the group trust?

3 What we did

Four parts, each run per configuration:

Part	What it does	Programs
RVFI replay on Spike	Records every retired instruction from the design's RVFI port, then replays the program on Spike. Interrupts and debug entries the design took are injected into Spike at the same instruction. Register writes, memory accesses and CSR reads are compared at every step.	All of the below
Seeded stimulus	A testbench patch that drives random interrupts, debug requests and instruction/data bus stalls from a seed, so a run can be repeated exactly	12 programs (checksum, muldiv, memcopy, callchain, sortbr, csrmix, fpmix on F/Z, and six of the core's own programs) x 20 seeds
The core's own directed programs	The test programs shipped with the core's verification repository, run on Verilator and replayed on Spike	31 to 34 programs per configuration, in two start-up layouts

Part	What it does	Programs
Random programs	riscv-dv Python generator (commit 919bfcf), 17 program classes from basic arithmetic to FP CSR and mstatus probes	Over 3,000 program runs

Every replay ends in one of three verdicts:

- MATCH: every instruction agrees.
- DIVERGED: first difference reported with pc, instruction, design value and Spike value.
- UNRESOLVED: the replay could not decide, for example a run too long to replay.

Known, legitimate differences between the design and Spike are declared, each with its reason, before they are allowed. The full list is under Limits.

4 Results

Under random interrupts, debug requests and bus stalls

Config	Runs	MATCH	Instructions compared	Interrupts followed	Debug entries followed
I0	240	240	23,007,800	111,333	27,339
F0	240	240	23,082,429	114,352	28,303
F1	60	60	5,776,107	28,859	7,234
F2	60	60	5,777,290	28,927	7,259
Z0	240	240	22,941,861	110,983	27,431
Z1	60	60	5,740,840	28,011	7,009
Z2	60	60	5,741,594	28,059	7,020
I0 stress (10x interrupt rate, stalls up to 15 cycles)	60	60	31,225,840	1,631,163	793,572
Total	1,020	1,020	123,293,761	2,081,709	905,367

Most of the interrupts (1.63 million of 2.08 million) and most of the debug entries (794,000 of 905,000) come from the I0 stress runs.

Can it see a bug at all? We applied five small RTL faults, one at a time, and reran. Every one was caught:

Fault patch	Detected at standard rates	Detected with targeted stimulus
mepc saved as pc+4	60 of 60 runs	-
dpc saved as pc+4 on a halt request	58 of 60	-
Load write-back dropped on an interrupt acknowledge	46 of 60	-
Interrupt 31 given priority below 30	9 of 60	15 of 15
Software interrupt given priority above external	4 of 60	15 of 15

The two priority faults only show when two interrupts are pending together, which random stimulus rarely creates. Targeted interrupt masks catch them every time, and the same masks on the unmodified design give 15 of 15 MATCH.

Random programs

	Result
Program runs	3,272, with 3,069,119 instructions compared, on I0, F0-F2, Z0-Z2. Of these, 1,554 are runs on the #1060-fixed design states.
Divergences	133, all in the fp_mstatus* classes on F0, F1 and F2 without a fix. That is #1060.
Everything else	MATCH, including every run on the fixed states

One correction to how these counts should be read. Some generated programs contain a random forward branch that jumps over the mstatus probe. Only 339 of 480 fp_mstatus probes actually execute. Every executed probe at distance 0 after flw reads stale on the unfixed design. See [report 02](#).

#1060

On ACT4 program priv/SmF/SmF-00 (F0), the replay stopped at the instruction where the bug occurs:

	Value
Instruction	csrrs x9, x0, 0x300 (read mstatus), at pc 0x1894
Design	0x00003880
Spike	0x80007880

Right after an flw, the design value has FS still Initial (01) instead of Dirty (11), and SD clear. The replay has no #1060-specific check, but this was not a blind find: we chose that program because of #1060 and predicted where it would diverge. With PR #1070 applied, the same program MATCHes over all 11,981 instructions.

Zfinx

22 comparison programs sweep every single-precision Zfinx instruction over a fixed operand table:

- signed zeros, subnormals, the largest finite values, infinities, quiet and signalling NaNs;
- all five rounding modes;
- fflags read after every operation.

Together with the 16 self-checking Zfinx programs, the results on Z0, Z1 and Z2 are 114 of 114 MATCH.

The core's own directed programs

Config	Programs evaluated	Self-check PASS (shipped / v1 start-up)	Replay MATCH	DIVERGED	UNRESOLVED
I0	31	23 / 28	27	4	2
F0, F1, F2 (each)	34	22 / 24	27	4	3
Z0, Z1, Z2 (each)	34	24 / 29	30	4	2

Results are identical across F0, F1 and F2, and across Z0, Z1 and Z2. Every program that doesn't pass has an identified cause, and none is an unexplained RTL difference.

Program	Cause
csr_instr_asm, illegal_instr_test	They report success by writing 1 to the status word, which the testbench decodes as failure. illegal_instr_test counted all 47,595 expected illegal instructions.
cv32e40p_csr_access_test, modeled_csr_por, requested_csr_por	Shipped start-up writes mtvec = 0x201; the programs expect the reset value. On F, the programs expect misa without F (0x40001104; the RTL reports 0x40001124).
debug_test	Expects tdata1 with the user-mode bit, which needs PULP_SECURE=1
debug_test_boot_set, debug_test_reset	Need a debug request at reset from the UVM environment
debug_test_trigger, debug_test_known_miscompares	Shipped start-up places the debugger stack outside the testbench's data window
interrupt_bootstrap, interrupt_test (F only)	mstatus.FS is Off after reset and the start-up code does not set it
matmul_32b_float (F only)	Enables the FPU only when PULP is defined, so every FP instruction traps. With FS enabled it passes on F0, F1 and F2.
hello-world (F, Z)	Expects mimpid 0 and the no-FPU misa
generic_exception_test (Z only)	Lists 11 instruction words with FP opcodes as illegal, but they are legal Zfinx instructions
cv32e40pv2_illegal_ro_csr_access_test, riscv_csr	Not buildable from any copy of the suite

Program	Replay verdict	Cause
all_csr_por	DIVERGED	Reads CSR 0x310 (mstatush). The design raises illegal instruction; Spike returns 0. See below.
cv32e40p_csr_access_test, isa_fcov_holes	DIVERGED	A defect in our replay for 32-bit values with bit 31 set, and for counters written then read
riscv_ebreak_test_0	DIVERGED	Our replay compares a store address as 64-bit; Spike sign-extends 0xccda4af8

Program	Replay verdict	Cause
coremark, matmul_32b_float (F)	UNRESOLVED	Runs of 10.2 and 18.4 million cycles, above our replay limit of 8 million
debug_test_trigger	UNRESOLVED	Debug entry by trigger match (cause 2) is not supported by our replay

mstatush (CSR 0x310). The RTL has no 0x310 case, so a read raises an illegal-instruction exception. The user manual states that the core implements Machine ISA version 1.11. mstatush belongs to version 1.12. This is a spec-version difference, not a defect. It is worth a line in the CSR list of the user manual, because a 1.12 toolchain or OS may read it.

5 What this means for TRL5

- It runs today, with no licence. It finds a real bug at the exact instruction, and it catches the interrupt, debug and load-write-back faults we planted.
- Its bug-finding power is not yet measured beyond the five faults above. We are measuring it with a blind fault-injection study across arithmetic, decode, CSR, interrupt and debug logic, and make no detection-rate claim until that is complete.
- Use it as a regression gate and a second opinion, not as the sign-off reference. The gaps below are real, and some of them (the debug cause, trigger entries) are exactly where ImperasDV's lock-step model is stronger.
- It does not resolve the reference-model question. It can't tell you whether the current Imperas model matches the frozen one. Disagreements between Spike and ImperasDV on the same trace would be a useful input to that question.
- The directed programs need repairs to the suite before they can count as tests. Two start-up layouts, the status-word convention, FS at reset, and two unbuildable programs.

6 How to repeat

Item	Version
Verilator	5.050
Spike	built from riscv-isa-sim (commit 02b1dc1), with the ISA string rv32imc_zicsr_zifencei_zicntr_zicclsm (with f added for F, _zfinx for Z)
riscv-dv	Python generator, commit 919bfcf
Testbench	cv32e40p-dv-review 89ad543 with the patches in patches/ (report 04)

The harness, stimulus patch and replay scripts are not public. The method needs only these pieces:

1. Expose the RVFI trace from simulation. The core already has bhv/cv32e40p_rvfi_trace.sv, so bind it in.
2. Log every retired instruction with its register write, memory access and CSR values.
3. Run the same ELF on Spike with --log-commits.
4. Compare step by step. When the design takes an interrupt or debug entry, inject the same event into Spike at that instruction.

We are happy to walk through it or run specific programs on request.

7 Limits

What this check cannot catch:

Gap	Effect
The debug-entry cause is taken from the design (dcsr.cause, or the RVFI debug flag), then given to Spike	A wrong debug cause is not flagged. ImperasDV computes the cause itself. This is the largest known gap.
Debug entry by trigger match	Not supported
Runs over 8 million cycles	Not replayed
Interrupt-priority faults	Caught reliably only with targeted interrupt masks; random rates catch them rarely
The microarchitectural window around the implicit FS-Dirty write (#1060)	Declared, because RVFI does not define mstatus.FS there. Explicit reads of mstatus are still compared, which is how #1060 is caught.

Declared differences: values the replay takes from the design instead of comparing, each with a reason:

- Machine identification CSRs and misa
- mtvec reset value and its WARL bits 7:2
- mcause WLRL bits 30:5
- Cycle and event counters
- Testbench peripheral loads
- mip, and the fast-interrupt bits of mie
- Trigger-module CSRs
- dcsr.stepie
- The FS window above

Not yet explained: in a later rerun of the unmodified design with the same tools, two results changed: one random Zfinx program (zfinx_arith, seed 7002, Z0) ended UNRESOLVED, and the replay of illegal_instr_test DIVERGED where the recorded run MATCHed. Both are being diagnosed; the counts above are from the recorded runs.

Scope:

- The operand tables and the stimulus programs were chosen by us. They are not an architectural test suite.
- RTL 6033d2b only, Verilator only, and the Verilator testbench memory model.

8 Files

- This report: cv32e40p-trl5/05-open-source-cross-check.md
- Testbench patches used: [patches/](#)
- Related: [report 02](#) for #1060, and [report 04](#) for the testbench patches